CAP 931 Student Process Guide

A practical workflow for building, testing, documenting, and sharing a sales agent prototype

This guide turns one completed CAP 931 project into a reusable learning reference. It explains how to move from the assignment brief to a working Streamlit prototype, evaluate its research, document the work honestly, and preserve the project with Git. Use it as a starting point; the current assignment brief and instructor directions always take priority.

The worked example researched wooden baseball bats for Target Corporation. It is useful because it includes both a successful iteration and mistakes that teach important lessons: irrelevant competitor inputs, unsupported product claims, missing buyer information, and the need to report uncertainty instead of guessing.

1. Understand the assignment before coding

The CAP 931 brief asks students to build a sales assistant prototype with a Streamlit interface and a GPT model. It should accept product and prospect information, research publicly available information, and produce a one-page account brief. The brief should cover company strategy, competitor mentions, relevant leadership, product or strategy context, and source links. The brief also asks students to document the technical choices, time allocation, challenges, experiments, and generated output using the Per Scholas report template. Alerts and production deployment are optional enhancements or plans.

The assignment brief (Version 1, dated September 19, 2024) does not explicitly require a GitHub repository link. This guide is based on that brief and the worked project's code, report, and logs; it is a student reference, not official Per Scholas policy. Follow the current brief, instructor, or submission portal if it gives additional requirements. GitHub is useful for version history, backups, and sharing.

Before implementation, make a checklist from the brief. Separate required features from optional ideas. A small, complete, tested prototype is easier to demonstrate and explain than an unfinished system with many extras.

2. Define a bounded minimum viable prototype

Write down the use case in one sentence: “Given a product and a prospective account, research public evidence and produce a short, source-linked sales brief.” Then define the minimum inputs, output, and tests.

For this project, the inputs were product name, product category, value proposition, prospective company and URL, competitor URLs, an optional contact, and an optional product overview file. The output was a Markdown and PDF brief with source links. The prototype did not implement alerts, public deployment, or persistent prospect storage.

The code used three sequential roles:

1. The Research Agent used web search to collect public information and source links.
2. The Account Analyst worked from the research packet to separate evidence, inference, and gaps.
3. The Brief Writer turned the analysis into a concise one-page brief.

Keeping the roles separate made the workflow easier to describe and review. It does not prove that a multi-agent design is more accurate or faster than one model call; that claim would require a controlled comparison.

3. Set up the project safely

The example used VS Code on macOS, Python 3.11, uv, Streamlit, and the OpenAI Responses API. Similar steps work in other editors, but shell commands may differ on Windows.

From the project root, the setup commands were:

```
uv sync
cp .env.example .env
```

Add the API key to the local `.env` file, then run the app:

```
uv run streamlit run app.py
```

The `.env` file contains a secret and must stay on the local computer. It should be ignored by Git. Commit a safe `.env.example` containing only placeholder text so another student knows which variable to configure. Never paste a live key into a report, screenshot, prompt, source file, or public repository.

Run the offline test suite with:

```
uv run pytest
```

The tests should not need a live API key or make network requests. In the worked example, all 10 automated tests passed.

4. Choose inputs that fit together

The product, category, value proposition, prospect, and competitor references should describe one coherent sales problem. Competitor URLs are research targets; they do not automatically establish that the prospect buys from, competes with, or partners with those companies.

For the worked example, the product was wooden baseball bats and the prospect was Target Corporation. The first run listed Snowflake and Databricks as competitor references. They are technology companies, so they were unrelated to the bat product. The second run used Marucci, Mizuno, and Victus, which better matched the product category.

Before a live run, ask:

- Do the competitor companies make or sell products in this category?
- Is the value proposition supported by product information, or is it just a marketing claim?
- Is the prospect URL the official public company site?
- Are optional fields truly optional, and are required fields clearly marked?

Use public business information. Do not enter private customer data unless the assignment and project have an approved way to handle it.

5. Build and test in small pieces

Keep the interface, pipeline, validation, document extraction, report generation, and tests in separate modules. In the example, `app.py` handled the Streamlit form; `src/sales_agent/pipeline.py` implemented the three roles; other modules validated URLs, extracted optional TXT/Markdown/PDF/DOCX text, and generated Markdown/PDF reports.

Validate inputs before spending money on model calls. The example checked required fields, public URL formats, duplicate competitor URLs, a five-URL competitor limit, and a 5 MB upload limit. API calls happened only after the user submitted the form.

Build a vertical slice first: collect inputs, run a minimal research path, display one result, then add source handling and downloads. Add tests for input validation, document parsing, and pipeline order. Run tests after meaningful changes and before demonstrating the app.

6. Review evidence and uncertainty, not just fluent writing

An LLM-generated sentence is not verified just because it sounds specific. Open the cited source and confirm that it supports the nearby claim. Check the source date, company identity, and whether the source is primary or secondary. Separate direct evidence from inference. If the research does not identify a buyer or establish a strategy, report “not found” or “unknown.”

The second Target run found wooden-bat listings, but a retail listing alone did not establish a strategic competitor relationship. One Victus listing was sold by a third party. The research did not establish that Target was prioritizing wooden bats or identify the category buyer. The product phrase “best wooden bat” remained an unsubstantiated claim. The brief treated these as limits instead of presenting them as facts.

Useful review questions:

- Does each important claim have a source that actually supports it?
- Is a source being mistaken for evidence of a partnership, purchase, or strategic priority?
- Are names, job titles, dates, and quotations accurate?
- Are unsupported claims labeled, removed, or converted into questions to verify?
- Is the brief concise enough to fit the one-page requirement?

The prototype uses web search through the model API. It does not directly fetch arbitrary URLs from the server and cannot guarantee that every supplied URL is indexed. Verify the links manually before using a brief with a real customer.

7. Keep an honest experiment log

Record each actual run as soon as it is reviewed. Capture the date, app or code version, model, exact inputs, prompt version when available, returned sources, ratings, unsupported claims, and the change planned for the next run. Do not invent measurements afterward. If time was not tracked live, label time estimates as retrospective estimates.

In the worked example, two runs kept the product, prospect, model (`gpt-6-luna`), and three-role workflow the same while changing the competitor references. The relevance rating rose from 2/5 to 4/5; grounding was rated 4/5 for both runs. These were retrospective qualitative ratings, not benchmark measurements. The exact prompt text and app commit for each run were not recorded.

The runs show that better-matched competitor inputs improved the perceived relevance of this example. They do not show that three agents outperform a single-pass prompt. No single-pass baseline, model comparison, cost benchmark, or response-time benchmark was completed. Do not claim comparative advantage without running a fair comparison using the same inputs, model, source checklist, and evaluation criteria.

8. Capture evidence while working

Capture genuine screenshots during the project rather than recreating them later. Useful evidence includes the project open in VS Code, environment or dependency setup, the app running, a completed input form, the generated brief with its sources, downloaded files, and the passing test output. Follow the actual report template and current instructor directions about which screenshots to include.

Check that screenshots are readable, show the relevant application state, and do not expose an API key, personal data, or unrelated desktop content. Keep the original output files as well as screenshots when the assignment requests the generated one-pager.

9. Write the report from project evidence

Use the supplied Per Scholas template and keep its required sections. A strong report explains what was built, how the code is organized, how to run it, why the model and workflow were selected, what the tests show, what failed, and what the output can and cannot support.

For the worked example, the project time log totaled 270 minutes (4.5 hours), but each entry was estimated retrospectively because exact time was not tracked. The report disclosed that limitation. The report also stated that production deployment had not occurred and described security, maintenance, and future improvements as plans rather than implemented features.

Use the generated brief and experiment log as evidence. Make the report consistent with the code: do not describe a feature as implemented if it is only proposed. Review the final PDF for blank pages, broken numbering, text split by screenshots, unreadable images, and leftover template instructions.

10. Use Git for version history

Initialize Git early, before major changes. A useful routine is to check status, test, review the staged file list, commit a coherent milestone, and push to a remote backup. Keep commits small enough to explain, such as setup, first working pipeline, validation, report generation, and final documentation.

Before committing, confirm that `.env`, virtual environments, caches, and local credentials are ignored. Review `git status` and the staged diff. The example established a clean `main` baseline commit after tests passed. It used a private GitHub repository; the assignment brief itself did not require a GitHub URL. A private repository is visible only to its owner and invited collaborators, so an instructor needs access if they are expected to review it.

11. Suggested final checklist

- The app starts from the documented command.
- Required fields and invalid URLs are handled clearly.
- The live run returns a concise brief and source links.
- Sources support the claims, and unknowns are marked.
- PDF and Markdown output open correctly.
- Automated tests pass without an API key.
- Actual experiments, limitations, and retrospective estimates are labeled honestly.
- The report uses the supplied template and includes requested evidence.
- `.env` and credentials are not committed or included in screenshots.
- Git status is clean after the final commit and the remote is verified.
- Submission files and access settings match the instructor's directions.

A model-neutral prompt for future students

Upload this guide together with your current assignment brief and project files. Then ask:

> *Use the assignment brief as the authority and this guide as a process reference. First extract the required deliverables and grading criteria. Compare them with my current project and identify missing evidence or requirements. Separate facts in my files from recommendations. Do not invent sources, test results, experiment scores, time logs, or implemented features. Guide me one step at a time and give commands for my operating system. Before any commit or public sharing, check for secrets and show me exactly what will be included.*